Review 1 Presentation on

Agentic Circuit Breakers:
Semantic Loop Detection and Cost Mitigation in Stateful Multi-Agent RAG Architectures

Submitted in partial fulfillment for the award of degree of

Bachelor of Technology

in

Department of Computer Engineering

By

Shreya Mahadik (121B1B150)
Harshal Patil (123B1B217)
Piyush Patil (123B1B220)
Sayali Pawar (123B1B229)

Under the supervision of

Prof. Sonika Gill

Assistant Professor

Pimpri Chinchwad College of Engineering, Pune

A. Y. 2026-27

Review 1: Outline

1. Problem Statement
2. Introduction & Motivation
3. Traditional vs Proposed Approach
4. Application Domain, SDG Mapping & Scope of Work
5. Research Objectives
6. Literature Review & Research Gaps
7. Proposed System Architecture
8. Mathematical Model & Technical Details
9. Conclusion & References

Motivation and Introduction

Research Problem Statement:

Design and implementation of a real-time, headless middleware — the Agentic Circuit Breaker — that detects semantic "thrashing" in stateful, LangGraph-style multi-agent RAG pipelines and forces graceful summarization instead of a hard, budget-exhausting crash.

Motivation:

- Multiple autonomous agents (Researcher, Critic, Writer) frequently enter a "recursive hallucination loop" — re-asking the same question in slightly different phrasing.
- Hard execution limits (max_iterations / recursion_limit) act as blunt trip-wires that crash the entire run.
- Observability platforms (LangSmith, AgentOps) only provide after-the-fact tracing — never a proactive, mid-run intervention.
- The 2026 MAESTRO study found 75.17% of multi-agent failures never even trigger a system exception.

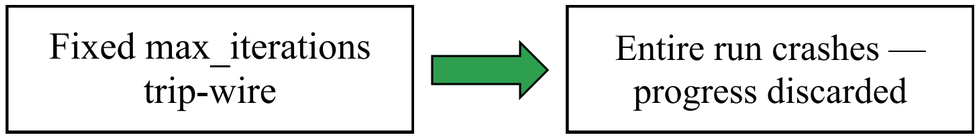
Figure: The Semantic Thrashing Loop

Introduction contd..

Traditional vs Proposed Approach

Traditional Approach

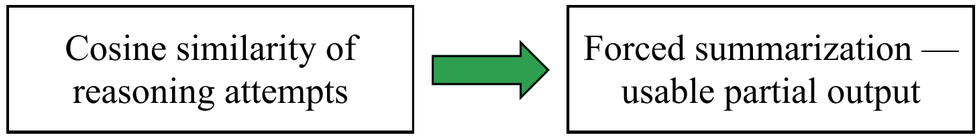
Fixed step budget — blunt trip-wire



- No awareness of whether the agent was one step away from converging.
- Purely reactive — numeric error / latency thresholds only (Hystrix-style).

Agentic Circuit Breaker (Proposed)

CLOSED → HALF-OPEN → OPEN state machine



- Proactively injects a system-level override to force summarization on detection.
- Returns usable partial output instead of crashing — graceful, cost-aware degradation.

Figure: Comparison of the Proposed Circuit Breaker with the Traditional Trip-Wire

Application Domain, SDG Mapping & Scope of Work

Application Domain: Software Development — AI Systems Reliability, Observability & Fault-Tolerance.

Sustainable Development Goal: SDG 9 — Industry, Innovation and Infrastructure. The project builds resilient, cost-efficient software infrastructure that keeps agentic AI industry deployments trustworthy at scale.

Scope of Research Work

- The system is engineered as a pure backend workflow with no UI overhead, operating natively via a terminal or JSON-driven API.
- Work covers a stateful, multi-hop query pipeline with three collaborating agents — Researcher, Critic, and Writer — querying a document corpus.
- Detection is intervention-triggered only, not an always-on redundant-consensus technique, keeping the approach cost-aware.
- The sliding-window comparison is limited to iteration N vs. iteration N-2 for judging convergence versus thrashing.
- The underlying pattern is built for LangGraph but generalises to other agent frameworks such as AutoGen and CrewAI, and is designed to sit alongside existing observability tools (LangSmith, AgentOps) rather than replace them.

Research Objectives

Objectives:

1. To identify and reliably reproduce the recursive-loop ("semantic thrashing") failure mode in stateful, LangGraph-style multi-agent RAG pipelines.

2. To design and implement a headless Middleware Interceptor that logs token counts, per-node embeddings, and state transitions without bloating the primary agent's context window.

3. To develop a Heuristic Engine — a CLOSED → HALF-OPEN → OPEN cosine-similarity state machine — that detects thrashing in real time over a sliding window of an agent's recent attempts.

4. To deliver a graceful-intervention mechanism (forced summarization or human-in-the-loop escalation) that returns usable partial output instead of a hard crash.

Literature Review

This work of Literature Review includes

1. Survey of real-time and post-hoc detection methods for multi-agent LLM failure and reasoning-loop thrashing.

2. Survey of resilience patterns, observability protocols, and token-budget / cost-overrun mitigation for agentic systems.

3. Survey of failure-attribution and multi-agent communication-structure analysis approaches.

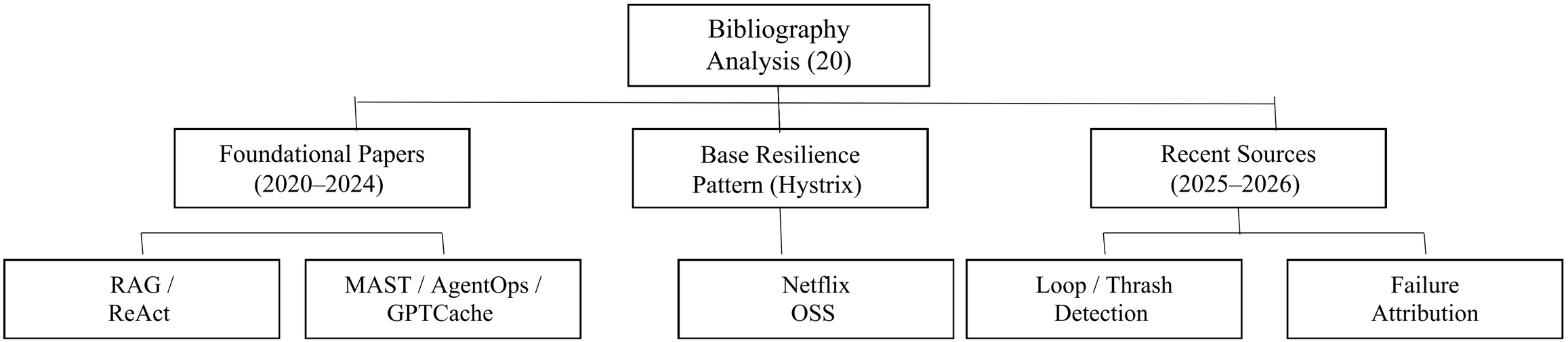


Figure 2: Bibliography Analysis for Literature Study (20 Sources, 2020–2026)

Literature Review — Foundational Papers

Table 1. Survey of RAG, Agentic Frameworks & Failure/Loop-Detection Studies

Sr.	Paper / Source	Method	Key Finding	Gap Identified
1	**Lewis et al., NeurIPS 2020 (RAG)**	Finds relevant information from documents and then generates an answer.	RAG performs very well for answering general questions.	It does not handle complex multi-step agent coordination or detect repeated/unhelpful searches.
2	**Yao et al., ICLR 2023 (ReAct)**	Combines AI reasoning with actions such as using tools and observing their results.	ReAct performs well on multi-step questions.	It mainly uses a fixed step limit, so an agent can keep repeating if it gets confused.
3	**Cemri et al., NeurIPS 2025 (MAST)**	Studied more than 1,600 failure traces from different multi-agent systems.	Identified 14 different types of multi-agent failures.	The failures are mainly analyzed after the execution; there is no real-time detector.
4	**FutureAGI Glossary, 2026**	Uses metrics such as loop rate, step efficiency, and token cost to study agent loops.	Explains the common causes of infinite loops in AI agents.	The metrics do not provide a mathematical way to detect the problem while the agent is running.
5	**C# Corner Case Study, 2026**	Uses a LangGraph workflow with a fixed max_iterations limit.	Shows reliable behaviour in a real-world fintech application.	It simply stops after reaching the limit and cannot understand whether the agent is actually close to completing the task.

Literature Review — Foundational Papers

Table 1. Survey of RAG, Agentic Frameworks & Failure/Loop-Detection Studies

Sr.	Paper / Source	Method	Key Finding	Gap Identified
6	**IAL-Scan, 2026**	Checks the agent's program/code structure to find possible infinite loops.	It is one of the first tools specifically designed to detect infinite loops in AI agents.	It works offline and cannot detect loops caused by real-time data or retrieved information.
7	**AgentOps, 2024**	Studies different monitoring tools and what information should be recorded from AI agents.	It explains what data should be logged for monitoring AI agents.	It does not automatically detect and stop a problem while the system is running.
8	**GPTCache, 2023**	Uses embeddings to find similar past queries and provide cached results.	It can make AI systems faster by reusing similar previous queries.	It compares queries from different users rather than monitoring one agent's own repeated behaviour.
9	**Netflix Hystrix, 2012–Present**	Uses a **CLOSED → OPEN → HALF-OPEN** circuit breaker based on errors and response time.	It is a well-tested method for making software systems more reliable.	It only checks numerical conditions like errors and delays; it does not understand the meaning of AI requests.
10	**FutureAGI Field Report, 2026**	Studies failures in real-world multi-agent AI systems.	It recommends using circuit breakers to stop problematic agents before they waste their budget.	It suggests the idea but does not explain exactly how to implement a semantic circuit breaker.

Literature Review — Recent Sources

Table 2. Newly Surveyed Papers on Runtime Loop Detection, Monitoring & Cost Overrun

Sr.	Paper / Source	Method	Key Finding	Gap Identified
11	**Shrivastava, 2026 – Semantic Early-Stopping**	Checks whether the meaning of an agent's recent outputs is becoming similar over time.	It can use embedding similarity to identify when an agent should stop.	It was tested only with 2-agent systems, not larger multi-agent systems.
12	**The Cognitive Companion, 2026**	Monitors the AI's internal behaviour while the main agent is working.	It can detect repeated or degraded reasoning without using another AI model as a judge.	It was mainly tested on small AI models, so its performance on larger systems is unclear.
13	**StepFinder, 2026**	Tracks the agent's reasoning steps using semantic embeddings to find where the failure happened.	It can identify the specific step where a failure occurs.	It mainly analyzes the problem after it happens and does not stop the problem in real time.
14	**Mesh Memory Protocol, 2026**	Provides shared memory and communication between agents during long-running tasks.	It helps different agents share information and work together for long periods.	It focuses on memory and communication, not detecting repeated loops.
15	**Causal Failure Attribution, 2025**	Uses causal analysis to find the important step that caused an agent failure.	It can identify the main step responsible for a failure.	It needs the complete execution trace and is not designed as a lightweight real-time detector.

Literature Review — Recent Sources

Table 2. Newly Surveyed Papers on Runtime Loop Detection, Monitoring & Cost Overrun

Sr.	Paper / Source	Method	Key Finding	Gap Identified
16	**Khan, 2026 – Token Budgets Catalog**	Studied real-world cases where AI agents used more tokens than expected.	Identified 63 cases of token-budget problems and grouped them into 8 types.	It focuses on controlling token budgets, not on detecting repeated AI reasoning.
17	**Shen et al., 2025 – Communication Topologies**	Studies how the way AI agents communicate affects information sharing.	Shows that agent communication structure affects the quality and efficiency of information flow.	It studies communication structure but does not provide a real-time system to stop failures.
18	**Wang et al., 2025 – AgentTaxo**	Studies how tokens are distributed among different AI agents and tasks.	Shows where most of the token usage occurs in a multi-agent system.	It only measures token usage and does not intervene when unusual or wasteful behaviour occurs.
19	**Owotogbe, 2025 – Chaos Engineering**	Intentionally introduces failures into multi-agent AI systems to test their reliability.	Helps evaluate and improve the system's ability to handle failures.	It tests failures but does not provide a semantic detector that can stop a loop during execution.
20	**Shigemura, 2025/26 – Recursive Knowledge Synthesis**	Uses three AI agents to check and verify information from different models.	Improves stability and explainability through repeated checking.	It adds another AI agent for checking, which increases the number of AI calls and overall cost.

Literature Review Analysis

Table 3. Thematic Synthesis of the 20 Surveyed Sources

Theme	No.	Representative Sources	Common Limitation
Retrieval & Reasoning Frameworks	2	RAG, ReAct	They use a fixed step limit and do not have a smart stopping method.
Failure Detection & Analysis	4	MAST, StepFinder, Causal Attribution, Chaos Engineering	They mainly find or analyze failures after they happen, not in real time.
Loop & Repetition Detection	4	FutureAGI, IAL-Scan, Semantic Early-Stopping, Cognitive Companion	They are either offline, limited, or mainly tested on two-agent systems.
Cost & Token Management	4	UPI Case Study, FutureAGI Field Report, Token Budgets Catalog, AgentTaxo	They track token usage and cost but do not check whether the AI is making meaningful progress.
Monitoring & Agent Communication	4	AgentOps, Mesh Memory Protocol, Communication Topologies, Recursive Knowledge Synthesis	They explain what to monitor or how agents communicate, but they don't provide a mechanism to stop a problematic loop.
Caching & Resilience	2	GPTCache, Hystrix	GPTCache focuses on similar queries, while Hystrix mainly uses numerical signals like errors and delays.

Synthesis: no surveyed source combines a real-time, stateful (Hystrix-style) trip mechanism with an LLM-native semantic signal — the gap this project targets.

Research Gaps

1. **No Real-Time Detection:** Existing methods cannot reliably detect during execution that an AI agent has stopped making progress.

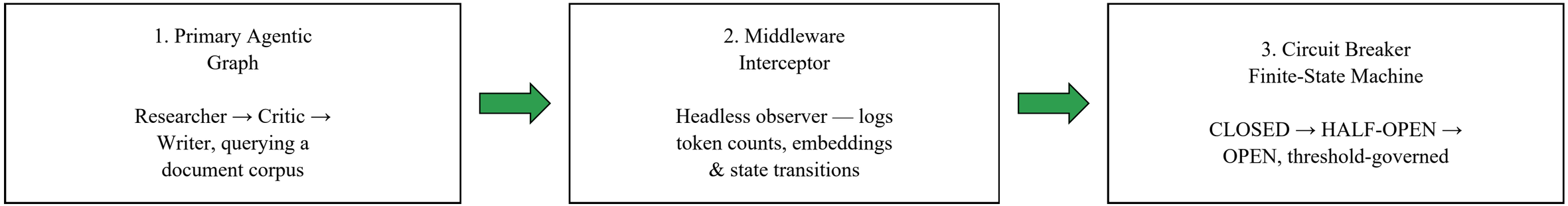
2. **Fixed Limits:** Limits like max_iterations and recursion_limit can stop or crash the entire system and lose useful progress.

3. **Semantic Caching:** Existing caching methods compare queries but do not track an agent's own repeated reasoning over time.

4. **Cost Control:** Token and cost monitoring measures spending but does not check whether the AI is making meaningful progress.

5. **No Combined Solution:** Existing systems do not combine semantic loop detection with the CLOSED → HALF-OPEN → OPEN circuit-breaker approach.

Proposed System Architecture



Forced summarization override injected back into the primary agentic graph on OPEN

End-to-End Flow:

1. Sequential reasoning / retrieval attempts are embedded and compared over a sliding window (iteration N vs N-2).
2. Cosine similarity ≥ 85% trips the breaker to HALF-OPEN; ≥ 95% trips it to OPEN (semantic thrashing confirmed).
3. On OPEN, the middleware forces summarization or escalates to a human-in-the-loop — backend-only, no UI overhead.

Figure 1: Agentic Circuit Breaker — System Architecture

Proposed System Architecture

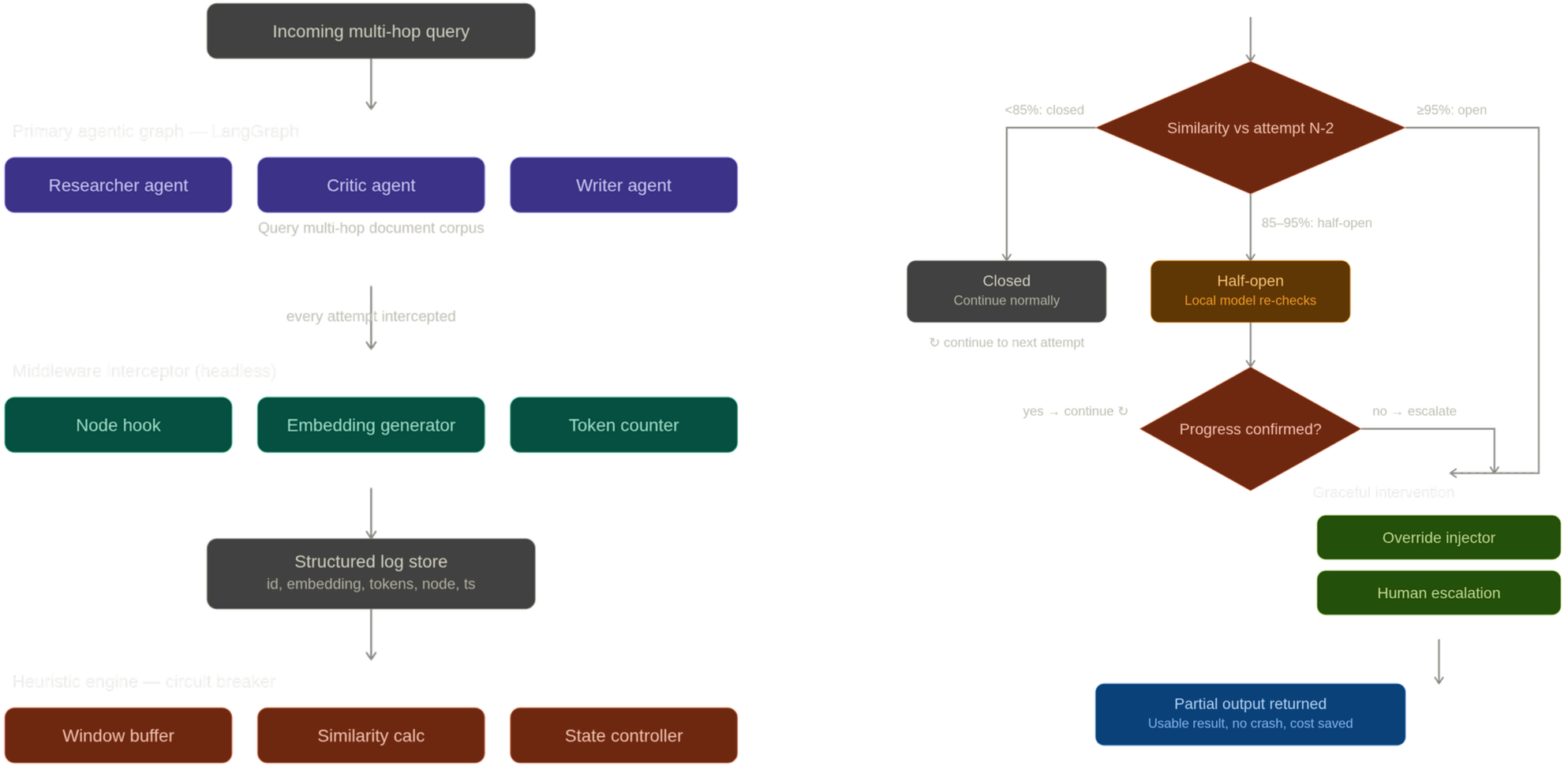


Figure 1: Agentic Circuit Breaker — System Architecture

Mathematical Model

Mathematical Formula Used:

(a) Cosine Similarity
Similarity between current and previous reasoning:

$$Sim(N, N-2) = (e_N \cdot e_{(N-2)}) / (||e_N|| \times ||e_{(N-2)}||)$$

Expanded form:

$$Sim(N, N-2) = \Sigma(e_{Ni} \times e_{(N-2)i}) / \sqrt{[\Sigma(e_{Ni}^2) \times \Sigma(e_{(N-2)i}^2)]}$$

(b) Threshold-Based State Transition

State = CLOSED, if $Sim < 0.85$

State = HALF-OPEN, if $0.85 \leq Sim < 0.95$

State = OPEN, if $Sim \geq 0.95$

c) Token Cost Savings

$$Savings (\%) = [(T_{baseline} - T_{treatment}) / T_{baseline}] \times 100$$

((d) Detection Accuracy
Precision:

$$P = TP / (TP + FP)$$

Recall:

$$R = TP / (TP + FN)$$

F1-Score:

$$F1 = 2PR / (P + R)$$

(e) Paired t-Test

$$t = \bar{d} / (s_d / \sqrt{n})$$

Where:

$\bar{d}$ = mean of per-question differences
s_d = standard deviation of differences
n = number of question pairs

Mathematical Model

Mathematical Formula Used:

(f) Effect Size — Cohen's d

$d = (\mu_1 - \mu_2) / \sigma\_pooled$

(g) One-Way ANOVA

$F = MS\_between / MS\_within$

Technology and Languages:

Python; LangGraph for the stateful multi-agent orchestration; an embedding model for computing sentence/reasoning-step vectors; cosine-similarity computation for the heuristic engine; a vector store for the retrieval corpus; a lightweight local model for the HALF-OPEN double-check step; and a terminal / JSON-driven API for the backend workflow (no UI overhead).

Conclusion

The Agentic Circuit Breaker addresses agent reliability as a real-time semantic problem rather than a purely numeric or post-hoc failure. By extending the proven Hystrix CLOSED → HALF-OPEN → OPEN state pattern with an embedding-based, LLM-native trip condition, the proposed middleware can detect and interrupt semantic thrashing before it leads to excessive resource and budget consumption, enabling graceful and cost-aware degradation instead of system failure. The approach fills a documented gap identified across the 20 surveyed sources, where no prior work combines a stateful resilience finite-state machine with a real-time semantic signal. Its lightweight, headless backend design introduces minimal context-window overhead while directly targeting budget-exhaustion failures reported in recent agentic-system incidents. Furthermore, although implemented with LangGraph, the approach is designed as a framework-agnostic resilience pattern that can be extended to other agent frameworks such as AutoGen and CrewAI.

References

[1] Lewis, P., Perez, E., Piktus, A., et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020. arXiv:2005.11401.

[2] Yao, S., Zhao, J., Yu, D., et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023. arXiv:2210.03629.

[3] Cemri, M., Pan, M.Z., Yang, S., et al. Why Do Multi-Agent LLM Systems Fail? (MAST). NeurIPS Datasets & Benchmarks, 2025. arXiv:2503.13657.

[4] FutureAGI. Infinite-Loop Agent Failure. FutureAGI Applied AI Glossary, 2026.

[5] Multi-Agent Reasoning Loops: Failure Modes & Mitigations in One-Tap UPI Payments. C# Corner (practitioner case study), 2026.

[6] When Agents Do Not Stop: Uncovering Infinite Agentic Loops in LLM Agents (IAL-Scan). arXiv:2607.01641, 2026.

[7] Dong, L., Lu, Q., Zhu, L. A Taxonomy of AgentOps for Enabling Observability of Foundation Model based Agents. arXiv:2411.05285, 2024.

[8] Bang, F. GPTCache: An Open-Source Semantic Cache for LLM Applications. OpenReview, 2023.

[9] Circuit Breaker Pattern as Implemented in Netflix Hystrix [BASE PAPER]. Netflix OSS, documented across Baeldung, DZone, Microsoft Learn, 2012–present.

[10] FutureAGI. Why Do Multi-Agent LLM Systems Fail? — Practitioner Field Report on Circuit Breakers and Token Budgets. FutureAGI Substack, 2026.

References

[11] Shrivastava, S. Semantic Early-Stopping for Iterative LLM Agent Loops. arXiv:2606.27009, 2026.

[12] The Cognitive Companion: A Lightweight Parallel Monitoring Architecture for Detecting and Recovering from Reasoning Degradation in LLM Agents. arXiv:2604.13759, 2026.

[13] StepFinder: A Temporal Semantic Framework for Failure Attribution in Multi-Agent Systems. arXiv:2606.03467, 2026.

[14] Mesh Memory Protocol: Semantic Infrastructure for Multi-Agent LLM Systems. arXiv:2604.19540, 2026.

[15] Ma, G., Zhu, J., Guo, H., et al. Automatic Failure Attribution and Critical Step Prediction Method for Multi-Agent Systems Based on Causal Inference. arXiv:2509.08682, 2025.

[16] Khan, S. Token Budgets: An Empirical Catalog of 63 LLM-Agent Budget-Overrun Incidents, with an Affine-Typed Rust Mitigation as a Case Study. arXiv:2606.04056, 2026.

[17] Shen, X., Liu, Y., Dai, Y., et al. Understanding the Information Propagation Effects of Communication Topologies in LLM-based Multi-Agent Systems. arXiv:2505.23352, 2025.

[18] Wang, Q., Tang, Z., Jiang, Z., et al. AgentTaxo: Dissecting and Benchmarking Token Distribution of LLM Multi-Agent Systems. ICLR 2025 Workshop on Foundation Models in the Wild.

[19] Owotogbe, J. Assessing and Enhancing the Robustness of LLM-based Multi-Agent Systems Through Chaos Engineering. IEEE/ACM CAIN, 2025, pp. 250–252.

[20] Shigemura, T. Recursive Knowledge Synthesis for Multi-LLM Systems: Stability Analysis and Tri-Agent Audit Framework. arXiv:2601.08839, 2025/26.

 THANK YOU